Módulo 6 - Testando e Automatizando Testes em Aplicações Web – Cypress

Nome: Mariana Caetano Costa
Email: mariana.cae.costa@gmail.com
Telefone: (31) 9.8859-0581

OBJETIVO DO MÓDULO

1. Objetivo

Ensinar a testar aplicações web por meio de testes funcionais e não funcionais, além de aprender a automatizar testes utilizando Cypress.

Ao longo das aulas, serão abordadas a configuração do projeto, a criação dos primeiros scripts de automação e o aprimoramento desses scripts, preparando o profissional de QA para as necessidades reais dos testes em aplicações web.

2. Resumo estruturado

Principais assuntos do módulo:

- Testes funcionais: verificam se as funcionalidades da aplicação se comportam conforme os requisitos esperados.
- Testes não funcionais: avaliam características de qualidade da aplicação, como acessibilidade, desempenho e responsividade.
- Automação de testes: utilização de scripts para executar verificações de forma automatizada.
- Configuração do projeto: preparação do ambiente para trabalhar com o Cypress.
- Criação de scripts: desenvolvimento dos primeiros testes automatizados.
- Melhoria dos scripts: aprimoramento dos testes para torná-los mais confiáveis e adequados às necessidades do projeto.

---

1. PREPARAÇÃO DO AMBIENTE

Durante o módulo, o ambiente utilizado para os testes foi composto pelos seguintes recursos:

Banco API

• Git Bash: acesso à pasta **banco-api**
• Inicialização do projeto pelo comando: **npm run rest-api**
• Acesso à aplicação pelo navegador.

Banco Web

• Git Bash: acesso à pasta **banco-web**
• Inicialização do servidor pelo comando: **npm run server**
• Acesso ao sistema pelo navegador.

Banco de Dados MySQL

• Banco de dados utilizado: banco.
• Usuário: julio.lima (acesso total)
• Usuário: junior.lima (acesso limitado)

---

2. MENTALIDADE DE TESTES EM APLICAÇÕES WEB

1. Objetivo da aula

Desenvolver a mentalidade necessária para testar aplicações web, compreendendo que a qualidade de um software não depende apenas de suas funcionalidades, mas também de fatores como compatibilidade, segurança, desempenho, usabilidade e acessibilidade.

O profissional de QA precisa analisar a aplicação sob diferentes perspectivas: a do usuário, a do servidor e a da estrutura tecnológica utilizada pelos desenvolvedores. Essa visão ampla permite identificar riscos e defeitos que poderiam prejudicar a experiência dos usuários.

2. Conceitos principais

2.1. Diversidade de usuários e ambientes

Aplicações web podem ser acessadas simultaneamente por pessoas de diferentes regiões do mundo, utilizando dispositivos, navegadores e versões distintas.

O QA precisa considerar:

- Dispositivos: celulares, computadores, tablets e outros equipamentos com navegador.
- Tamanhos de tela: telas pequenas, médias e grandes.
- Navegadores: Chrome, Edge, Firefox, Safari e outros.
- Versões dos navegadores: versões diferentes podem apresentar comportamentos distintos.
- Acesso simultâneo: vários usuários podem utilizar a aplicação ao mesmo tempo.

Essa diversidade cria diferentes condições de uso que precisam ser consideradas durante os testes.

2.2. Características de qualidade das aplicações web

O teste de aplicações web deve considerar diferentes características de qualidade.

Característica	**O que o QA deve avaliar**
Compatibilidade	Se a aplicação funciona corretamente em diferentes navegadores, versões e dispositivos.
Responsividade	Se a interface se adapta adequadamente a diferentes tamanhos de tela.
Segurança	Se a aplicação está protegida contra acessos indevidos e ações maliciosas.
Desempenho	Se a aplicação responde adequadamente e mantém um bom desempenho.
Usabilidade	Se a aplicação é fácil de entender e utilizar.
Navegabilidade	Se o usuário consegue encontrar informações e transitar entre as páginas com facilidade.
Acessibilidade	Se pessoas com diferentes necessidades conseguem utilizar a aplicação.

Essas características ajudam o QA a identificar o que precisa ser avaliado em cada projeto e funcionam como um checklist para orientar a execução dos testes.

2.3. Mentalidade orientada a riscos e defeitos

O trabalho do QA não deve se limitar a verificar se o sistema funciona em uma situação ideal.
É necessário pensar em diferentes cenários de uso, identificar possíveis falhas e investigar situações que possam comprometer a qualidade do software.
O objetivo é revelar o maior número possível de riscos e defeitos relevantes, contribuindo para a entrega de um software confiável e adequado às necessidades dos usuários.

3. Compreensão da arquitetura de uma aplicação web

Para testar aplicações web de maneira eficiente, o QA precisa compreender como suas principais partes se comunicam.



Front-end: é a parte com a qual o usuário interage diretamente. O QA verifica se os elementos da interface funcionam corretamente e se as informações são apresentadas de maneira adequada.

Back-end: é a parte responsável pelo processamento das informações e pela execução de regras de negócio. Ele recebe requisições, processa os dados e devolve respostas ao front-end.

Comunicação entre front-end e back-end: o QA precisa verificar se essas partes estão se comunicando corretamente, inclusive quando ocorrem erros.

Exemplo prático: ao realizar um login, o front-end envia as credenciais para o back-end. O QA deve verificar tanto o comportamento da interface quanto o tratamento da resposta recebida.

Entre os cenários possíveis estão:

- Login com credenciais válidas.
- Login com senha incorreta.
- Resposta de erro do servidor.
- Falha de comunicação com o back-end.
- Exibição correta das mensagens de sucesso ou erro.

O sistema deve tratar adequadamente tanto as respostas de sucesso quanto as situações de falha.

4. Conhecimento técnico necessário para o QA

Além de compreender o comportamento da aplicação, é importante conhecer os conceitos e as tecnologias utilizadas em seu desenvolvimento.

Tecnologia ou conceito	**Importância para o QA**
HTML	Compreender a estrutura dos elementos da página.
CSS	Entender a apresentação visual e os estilos da interface.
JavaScript	Compreender comportamentos e interações dinâmicas.
React	Reconhecer conceitos de componentes e interfaces construídas com essa biblioteca.
Angular	Compreender a estrutura de aplicações desenvolvidas com esse framework.
Componentes	Entender como os elementos da interface são organizados e funcionam.

Não significa que o QA precise necessariamente desenvolver toda a aplicação. O objetivo é adquirir conhecimento técnico suficiente para entender sua estrutura, identificar possíveis problemas e elaborar testes mais eficientes.
O conhecimento das tecnologias também ajuda a compreender como os componentes são configurados, como se comportam e como interagem com os dados recebidos do servidor.

5. Como aplicar essa mentalidade nos testes
Ao receber uma aplicação web para testar, o QA deve pensar além do caminho principal de utilização.

Por exemplo, ao testar um formulário de cadastro, pode investigar:

- O formulário funciona em diferentes navegadores?
- A interface se adapta a telas menores?
- Os campos validam corretamente os dados?
- O sistema apresenta mensagens adequadas quando os dados são inválidos?
- O que acontece quando o servidor retorna um erro?
- A aplicação continua funcionando quando há muitos acessos simultâneos?
- Os elementos podem ser utilizados por pessoas que dependem de recursos de acessibilidade?

Essas perguntas ajudam a ampliar a cobertura dos testes e a identificar riscos que poderiam passar despercebidos em uma verificação superficial.

6. Pontos essenciais para memorizar

- Testar aplicações web exige considerar diferentes usuários, dispositivos, navegadores e versões.
- A qualidade não se limita às funcionalidades: envolve compatibilidade, segurança, desempenho, usabilidade, navegabilidade e acessibilidade.
- O QA deve compreender a diferença entre front-end e back-end e verificar a comunicação entre eles.
- É importante conhecer as tecnologias utilizadas na construção da aplicação, como HTML, CSS, JavaScript, React e Angular.
- Os testes devem contemplar cenários de sucesso, falha e condições variadas de utilização.
- Uma mentalidade orientada a riscos ajuda a identificar defeitos e a entregar um software de melhor qualidade.

Ideia central da aula: testar aplicações web significa avaliar o software sob diferentes perspectivas — a do usuário, a da comunicação com o servidor e a da estrutura técnica — para identificar riscos e defeitos e melhorar a qualidade do produto.

---

3. TESTES DE ACESSIBILIDADE

1. Objetivo da aula
Compreender a importância dos testes de acessibilidade digital, aprender a identificar problemas que dificultam o uso de aplicações web por pessoas com deficiência e conhecer a ferramenta Axe DevTools, utilizada para detectar problemas de acessibilidade.

A aula também demonstra como instalar a extensão no Google Chrome, executar uma análise de acessibilidade, interpretar os resultados e encaminhar os defeitos encontrados para correção. Por fim, apresenta a possibilidade de integrar o Axe ao Cypress para automatizar verificações de acessibilidade.

2. Conceitos principais

2.1. Acessibilidade digital

Acessibilidade digital consiste em avaliar se sites, aplicativos e outros softwares permitem que todas as pessoas consigam acessar, compreender e interagir com seus conteúdos e funcionalidades.

Seu objetivo está diretamente relacionado à inclusão digital, permitindo que pessoas com diferentes necessidades utilizem aplicações com autonomia.

Por exemplo, uma pessoa cega deve conseguir acessar um site bancário, consultar informações de sua conta e realizar operações sem depender necessariamente de outra pessoa.
O QA deve identificar barreiras de acessibilidade para que os desenvolvedores possam corrigi-las.

2.2. Leitores de tela

Leitores de tela são recursos assistivos que interpretam os elementos de uma interface e comunicam suas informações ao usuário por meio de áudio.
Uma pessoa pode utilizar a tecla Tab para navegar entre os elementos de uma página. O leitor de tela informa qual elemento está em foco, desde que existam informações acessíveis que permitam identificá-lo. Quando um campo ou botão não possui uma identificação adequada, o leitor de tela pode não conseguir comunicar corretamente sua finalidade.
Por isso, o QA deve verificar se os elementos da interface possuem nomes acessíveis e se podem ser utilizados por pessoas que dependem de tecnologias assistivas.

2.3. Testes funcionais e testes de acessibilidade

Os testes funcionais verificam se as funcionalidades se comportam conforme o esperado. Já os testes de acessibilidade avaliam se essas funcionalidades podem ser utilizadas por pessoas com diferentes necessidades.

Por exemplo, um botão de transferência bancária pode funcionar corretamente com o mouse, mas apresentar problemas para uma pessoa que utiliza leitor de tela ou navegação pelo teclado.
Portanto, uma aplicação funcionar não significa que ela seja acessível.

3. Ferramenta Axe DevTools

O Axe DevTools é uma ferramenta que auxilia na identificação de problemas de acessibilidade em páginas web. Na aula, o instrutor utiliza a extensão para Google Chrome em sua versão gratuita.

A ferramenta analisa a página e apresenta problemas encontrados, suas descrições, os elementos afetados e orientações para investigação e correção.

O instrutor demonstra dois exemplos importantes de problemas:

- Contraste de cores insuficiente: textos e elementos visuais podem ficar difíceis de enxergar.
- Campos de formulário sem identificação adequada: determinados campos podem não possuir rótulos (labels) que permitam compreender sua finalidade.

O Axe também pode ser integrado a ferramentas de automação, como o Cypress, para executar verificações de acessibilidade durante testes automatizados.

4. Como instalar e utilizar o Axe DevTools

O procedimento demonstrado na aula é o seguinte:

1. Acesse a Chrome Web Store pelo Google Chrome.
2. Pesquise por Axe DevTools e selecione a extensão correspondente.
3. Clique em Adicionar ao Chrome e confirme a instalação.
4. Abra a aplicação web que será testada.

5. Clique com o botão direito na página e selecione Inspecionar para abrir as ferramentas de desenvolvedor.
6. Localize a aba Axe DevTools. Caso não esteja visível, procure-a no menu de abas adicionais.
7. Selecione a opção de análise completa da página (Full Page Scan).
8. Execute a análise e examine o relatório de problemas encontrados.
9. Investigue cada problema, identifique os elementos afetados e registre os defeitos que precisam ser corrigidos.

Se a análise não funcionar corretamente porque a página ainda não foi carregada por completo, o instrutor recomenda atualizar a página e executar o procedimento novamente.

5. Como interpretar os resultados da análise

Durante a demonstração, o instrutor executa o Axe DevTools em uma tela de transferência bancária e encontra oito problemas de acessibilidade: dois classificados como críticos e seis como sérios.
Esses números correspondem exclusivamente à aplicação e à execução demonstradas na aula, não a uma quantidade fixa de problemas que a ferramenta sempre encontrará.

Exemplo de interpretação do relatório:

Problema identificado	**O que significa**	**O que o QA deve fazer**
Contraste insuficiente	A diferença entre a cor do texto e o fundo pode dificultar a leitura.	Identificar os elementos afetados e registrar a violação.
Campo sem label	Um campo pode não possuir um rótulo que descreva sua finalidade.	Verificar a identificação acessível do campo e registrar o problema.
Problema crítico	A ferramenta atribuiu alta severidade à ocorrência.	Priorizar a investigação e a correção.
Problema sério	A ocorrência também pode prejudicar a acessibilidade.	Planejar a correção conforme a severidade, o impacto e as prioridades do projeto.

O relatório permite visualizar a página analisada, consultar as descrições das ocorrências e identificar os elementos específicos que precisam ser investigados.

Prioridade de correção: o instrutor recomenda começar pelos problemas críticos e, em seguida, tratar os problemas sérios. Na prática, a priorização também deve considerar o impacto para os usuários, o contexto da aplicação e os critérios definidos pela equipe.

6. Contraste de cores e WCAG

A aula apresenta as diretrizes WCAG (Web Content Accessibility Guidelines), que estabelecem recomendações para tornar conteúdos web mais acessíveis.

Um dos aspectos avaliados é o contraste entre a cor do texto e a cor do fundo.

Na demonstração, o Axe identifica um elemento com contraste de 2,67:1, enquanto a regra apresentada exige um mínimo de 4,50:1 para aquele caso.

Isso significa que o contraste identificado não atende ao limite aplicado pela ferramenta.

O relatório também indica qual elemento apresenta o problema, facilitando a investigação pelo QA e a correção pelos desenvolvedores.

É importante observar que os limites de contraste variam conforme o tipo e o tamanho do texto e o critério WCAG aplicável.

7. Labels nos elementos de formulário

Um label é um rótulo que identifica a finalidade de um campo de formulário.

Em uma tela bancária, exemplos de campos são:

- Conta de origem.
- Conta de destino.
- Valor da transferência.
- Token.

Na aula, o Axe identifica campos que não possuem labels adequados, incluindo elementos do tipo select e input.

O QA deve verificar se os campos podem ser identificados corretamente, inclusive por leitores de tela.

A identificação acessível pode ser implementada de diferentes formas, não necessariamente apenas por um elemento visual <label>. O importante é que o controle possua um nome acessível adequado à sua finalidade.

8. Como registrar os defeitos encontrados

Após executar a análise, o QA deve comunicar os problemas identificados à equipe de desenvolvimento.

Conforme explicado pelo instrutor, isso pode ocorrer por meio de uma conversa com os desenvolvedores ou pela abertura de defeitos na ferramenta de gestão utilizada pela equipe.

Um registro de defeito deve conter informações que ajudem a reproduzir e compreender o problema, como:

- Página e elemento afetados.
- Descrição do problema de acessibilidade.
- Resultado encontrado na análise.
- Critério ou regra violada.
- Evidências, quando disponíveis.
- Impacto para o usuário.
- Prioridade ou severidade definida pela equipe.

O objetivo é fornecer informações suficientes para que o defeito seja compreendido, corrigido e posteriormente validado.

9. Integração do Axe com Cypress

O instrutor explica que o Axe não se limita à análise manual por meio da extensão do navegador.

Também é possível utilizar uma biblioteca de integração com o Cypress para executar verificações de acessibilidade dentro de testes automatizados.

Com essa integração, o QA pode incluir verificações de acessibilidade nos cenários automatizados, facilitando a identificação recorrente de problemas nas páginas testadas.

Por exemplo, em um teste automatizado de transferência bancária, é possível verificar se a tela apresenta violações de acessibilidade detectáveis pelas regras configuradas.

Essa abordagem complementa os testes funcionais, permitindo avaliar diferentes aspectos da qualidade da aplicação durante a automação.

Vale lembrar que ferramentas automatizadas detectam apenas determinadas categorias de problemas. Elas não substituem testes manuais, navegação por teclado, avaliação com leitores de tela e outras verificações de acessibilidade.

10. Pontos essenciais para memorizar

- A acessibilidade digital busca permitir que todas as pessoas acessem, compreendam e utilizem aplicações.
- Leitores de tela auxiliam pessoas cegas a interagir com páginas web.
- O QA deve verificar a acessibilidade além do funcionamento das funcionalidades.
- O Axe DevTools identifica problemas de acessibilidade e apresenta orientações para investigá-los.
- Contraste insuficiente e campos sem identificação acessível são exemplos de problemas que podem ser detectados.
- WCAG é o conjunto de diretrizes de acessibilidade para conteúdos web.
- Os defeitos encontrados devem ser documentados e encaminhados para correção.
- O Axe pode ser integrado ao Cypress para automatizar verificações de acessibilidade.
- A automação auxilia na detecção de problemas, mas não substitui uma avaliação completa de acessibilidade.

Ideia central da aula: o trabalho do QA também envolve verificar se uma aplicação pode ser utilizada por pessoas com diferentes necessidades. O Axe DevTools ajuda a identificar barreiras de acessibilidade, e sua integração com o Cypress permite incorporar verificações ao processo de automação.

---

4. TESTES DE PERFORMANCE DO FRONTEND

1. Objetivo da aula

Aprender a avaliar a performance de uma aplicação web utilizando o Lighthouse, ferramenta integrada ao Google Chrome que analisa o carregamento e a renderização das páginas.

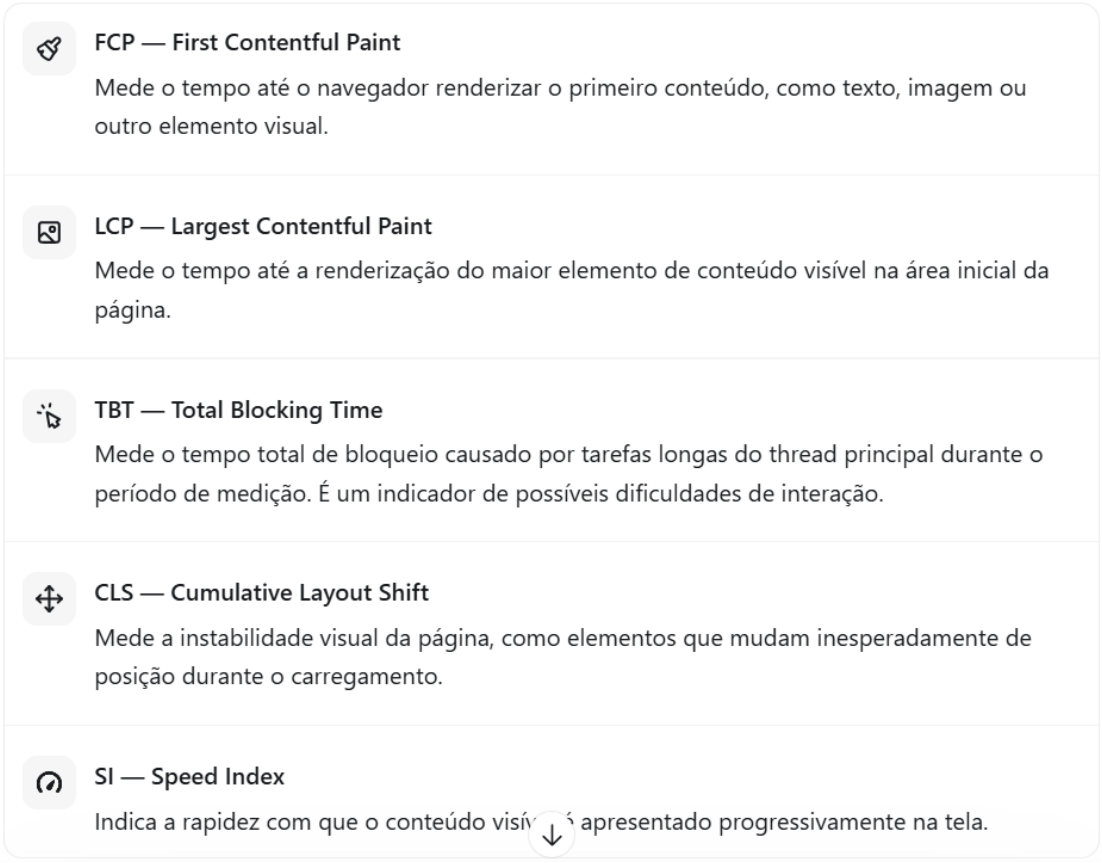
O objetivo do QA é identificar problemas de desempenho, reunir evidências e compartilhar oportunidades de melhoria com os desenvolvedores.

2. Como executar o Lighthouse

1. Abra a aplicação web no Google Chrome.
2. Clique com o botão direito na página e selecione Inspecionar (Inspect).
3. Acesse a aba Lighthouse no DevTools.
4. Configure a análise:
    - Mode: Navigation.
    - Categories: somente Performance.
    - Device: Desktop.
5. Clique em Analyze page load.
6. Analise a pontuação, as métricas, os diagnósticos e as oportunidades de melhoria.

A aula recomenda executar o teste em um ambiente semelhante ao de produção, pois os resultados locais podem ser melhores do que os encontrados em condições reais.

3. Principais métricas de performance



4. Como interpretar os resultados

O Lighthouse apresenta uma pontuação geral de performance e os valores das métricas avaliadas. A pontuação é influenciada por métricas e pesos definidos pela ferramenta.

Na aula, o site analisado recebeu nota 100, com resultados muito bons. O instrutor explica que executar o teste localmente pode favorecer os resultados em comparação com um ambiente de produção.

Além das métricas, o relatório apresenta:

- Filmstrip: imagens que mostram a evolução visual do carregamento.
- Diagnostics: informações técnicas sobre possíveis problemas.
- Opportunities: sugestões de otimização, geralmente acompanhadas de estimativas de economia de tempo ou de bytes.
- Passed audits: verificações aprovadas.

5. Exemplos de melhorias que podem ser identificadas

- Eliminar recursos que bloqueiam a renderização: avaliar CSS e JavaScript que atrasam a exibição do conteúdo.
- Minificar JavaScript: reduzir o tamanho dos arquivos removendo caracteres desnecessários sem alterar seu comportamento.
- Investigar tarefas longas: reduzir o bloqueio do thread principal.
- Corrigir instabilidade visual: evitar que elementos mudem inesperadamente de posição.

As economias estimadas pelo Lighthouse são oportunidades potenciais, não garantias de que o ganho será exatamente aquele valor.

6. Qual é o papel do QA nesse processo?

O QA pode utilizar o Lighthouse para:

1. Executar a análise de performance.
2. Registrar as métricas e a pontuação obtidas.
3. Identificar oportunidades de melhoria.
4. Coletar evidências do relatório.
5. Comunicar os achados ao time de desenvolvimento.
6. Reexecutar o teste após as correções e comparar os resultados.

A ferramenta também pode ser utilizada em processos seletivos e testes técnicos para demonstrar conhecimentos além dos testes funcionais tradicionais.

7. Aplicação prática para seus estudos de QA

Para transformar o conteúdo da aula em prática profissional, registre cada execução com:

Campo	O que registrar
Aplicação	Nome e versão testada
Ambiente	Local, homologação ou produção
Configuração	Navigation, Performance e Desktop
Resultado	Pontuação e valores das métricas
Evidências	Relatório, capturas de tela e diagnósticos
Achados	Problemas ou oportunidades identificados
Reteste	Comparação antes e depois da correção

Atenção ao seu erro anterior: o NO_FCP significa que o Lighthouse não conseguiu detectar o primeiro conteúdo renderizado naquela execução. Antes de tirar conclusões sobre a performance, verifique se a página carregou corretamente e repita o teste em condições adequadas.

O que você precisa guardar desta aula

- O Lighthouse é uma ferramenta para avaliar a performance do frontend.
- FCP e LCP avaliam tempos de renderização de conteúdo.
- TBT avalia bloqueios causados por tarefas longas.
- CLS avalia a instabilidade visual.
- Speed Index avalia a velocidade de apresentação do conteúdo.
- O relatório ajuda a identificar oportunidades de otimização.
- O QA deve documentar os resultados, apresentar evidências e comparar as medições após as correções.

Principal aprendizado: testar performance não é apenas obter uma nota. É medir o comportamento da aplicação, identificar possíveis gargalos e fornecer informações que ajudem o time a melhorar a experiência do usuário.

---

5. TESTES DE RESPONSIVIDADE

1. O que são testes de responsividade?

A responsividade é a capacidade de uma aplicação web de adaptar seu layout aos diferentes tamanhos de tela, mantendo a organização dos elementos e uma boa experiência para o usuário.

A aplicação pode ser acessada por notebooks, tablets, smartphones e monitores grandes. Os elementos devem se reorganizar conforme as dimensões disponíveis, e não simplesmente ficar comprimidos ou sobrepostos.

2. Como funciona a responsividade?

O desenvolvedor pode implementar regras que alteram o posicionamento dos elementos de acordo com a largura da tela. Frameworks também podem oferecer recursos que facilitam essa implementação, mas é necessário verificar se foram configurados corretamente.

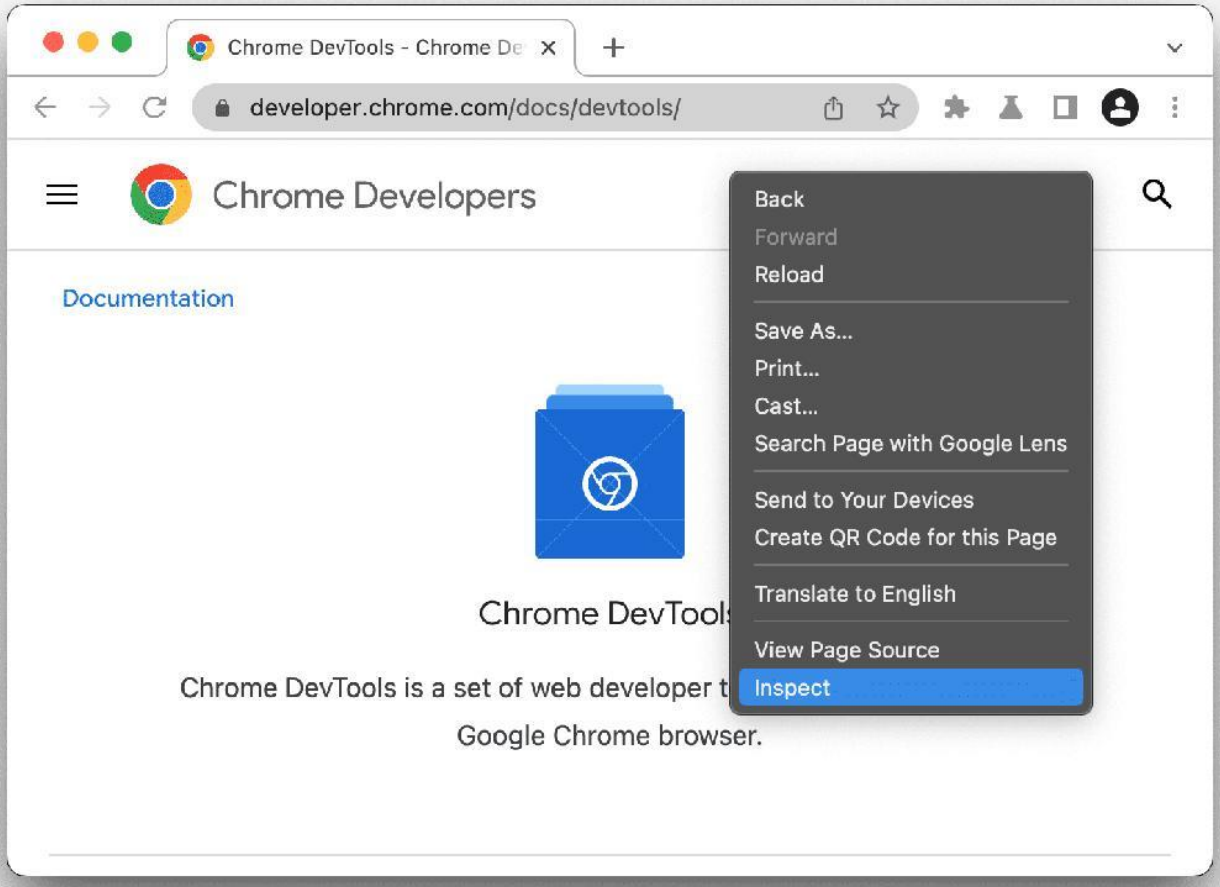
Exemplo apresentado na aula:

- Até 992 px: logomarca centralizada.
- A partir de 993 px: logomarca posicionada à esquerda.

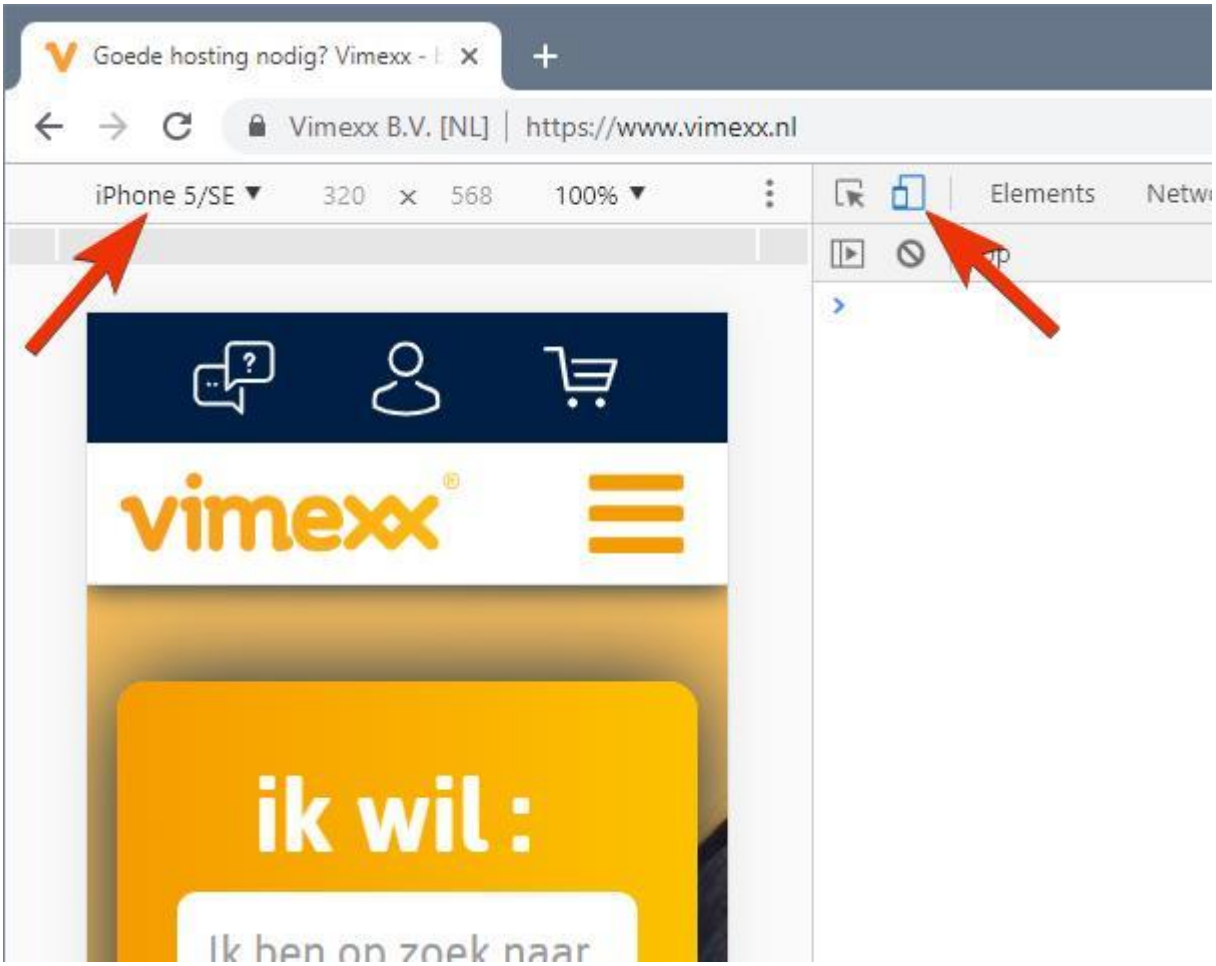
Esses valores são apenas o exemplo da aplicação demonstrada na aula, não uma regra universal para todos os sites.

3. Ferramenta utilizada: Chrome DevTools

O Google Chrome possui ferramentas integradas para testar a responsividade sem precisar de um dispositivo físico.

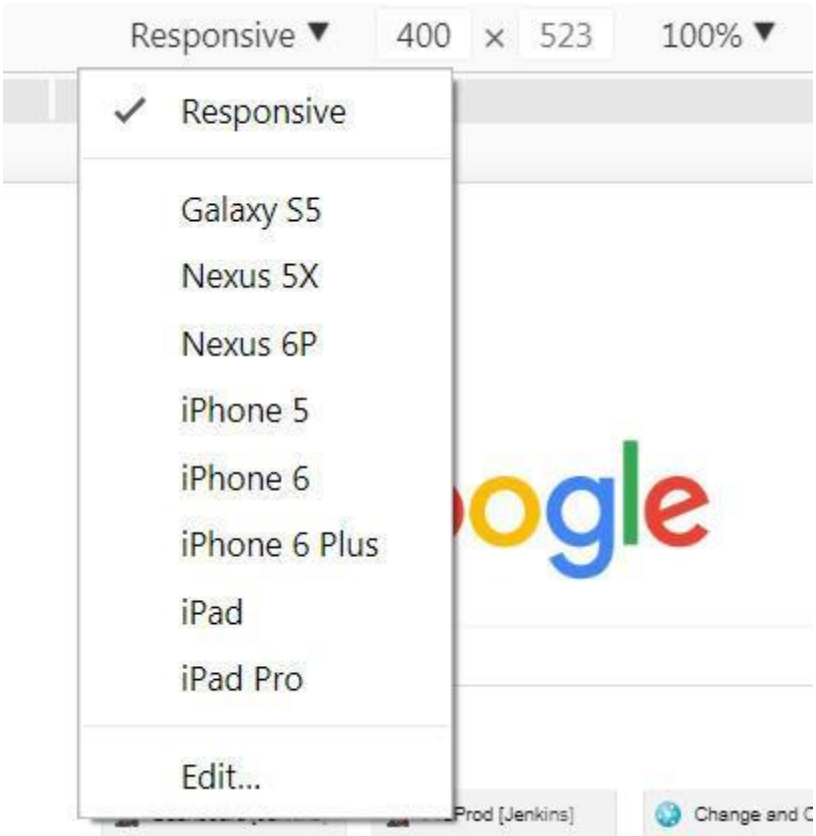


1. Abrir as ferramentas de desenvolvedor
Clique com o botão direito na página e selecione Inspecionar (Inspect).
Alternativamente, acesse o menu de três pontos → Mais ferramentas → Ferramentas do desenvolvedor.



2. Ativar a simulação de dispositivos

Clique no ícone de dispositivo móvel/computador (Toggle device toolbar) e selecione Responsive para ajustar manualmente a largura e a altura.



3. Selecionar um dispositivo
Escolha um dispositivo disponível, como um iPhone, para simular suas dimensões e verificar como a página é apresentada.

4. O que o QA deve testar?

- Se os elementos mudam de posição nos tamanhos de tela esperados.
- Se textos, imagens, botões e menus continuam visíveis e utilizáveis.
- Se existem elementos sobrepostos, cortados ou fora da tela.
- Se mensagens de erro e avisos aparecem na posição correta.
- Se os comportamentos correspondem ao layout especificado pelo designer ou à documentação dos desenvolvedores.
- Se a aplicação continua funcional em diferentes larguras e dispositivos.

5. Exemplo prático de teste

Imagine que a documentação informe que a logomarca deve ficar centralizada em telas de até 992 px e à esquerda em telas a partir de 993 px.

Largura da tela	**Resultado esperado**
390 px	Logomarca centralizada
768 px	Logomarca centralizada
992 px	Logomarca centralizada
993 px	Logomarca à esquerda
1440 px	Logomarca à esquerda

Se a logomarca estiver à esquerda em 992 px, por exemplo, existe uma divergência em relação à regra especificada.

6. Conceitos importantes para memorizar

- Responsividade: adaptação do layout a diferentes dimensões de tela.
- Breakpoint: largura na qual uma regra de layout passa a valer ou muda.
- Chrome DevTools: ferramenta do navegador que permite inspecionar e simular diferentes dispositivos.
- Renderização: forma como os elementos visuais são apresentados na tela.
- Validação do QA: comparar o comportamento observado com o comportamento esperado e documentado.

Principal aprendizado: o QA deve testar a aplicação em diferentes dimensões de tela, verificar a reorganização dos elementos e comparar os resultados com as especificações do projeto. A ferramenta do Chrome facilita esse processo sem exigir vários dispositivos físicos.

---

6. FAZENDO TESTES ISOLADOS USANDO MOCKS

1. O que são testes isolados usando mocks?
Em aplicações web, o frontend geralmente se comunica com uma API, que disponibiliza funcionalidades do backend.

Como o desenvolvimento do frontend e do backend pode ocorrer em paralelo, uma parte pode ficar pronta antes da outra. Para não precisar esperar a integração completa, o QA pode utilizar mocks para simular as respostas da API e começar a testar antecipadamente.

Mock: simulação de um componente ou serviço, utilizada para reproduzir comportamentos esperados sem depender da implementação real.

2. Por que utilizar mocks?

- Antecipar a execução dos testes.
- Identificar defeitos antes da conclusão do backend.
- Testar o frontend de forma independente.
- Simular respostas de sucesso e erro.
- Reduzir a dependência entre as equipes de desenvolvimento.

3. Ferramenta apresentada: Mockoon
O instrutor apresenta o Mockoon, ferramenta utilizada para criar APIs simuladas.

O procedimento demonstrado na aula é:

1. Baixar e instalar o Mockoon.
2. Abrir a aplicação e configurar um ambiente.
3. Definir a porta do servidor, utilizando uma porta disponível.
4. Criar uma rota HTTP.
5. Configurar o método HTTP, o endpoint e as respostas esperadas.
6. Executar o servidor mock para que o frontend possa consumi-lo.

4. Exemplo prático: teste de login
A aula utiliza uma tela de login para demonstrar a comunicação entre frontend e backend.

A requisição apresentada possui as seguintes características:

Item	**Configuração**
Método HTTP	POST
Endpoint	/login
Dados enviados	username e senha
Resposta esperada	Token de autenticação
Sucesso	Status HTTP da família 2xx
Erro	Tratamento e apresentação de mensagem de erro

O fluxo demonstrado é:

1. O usuário informa o nome de usuário e a senha.
2. O frontend envia uma requisição à API.
3. A API processa a solicitação e retorna uma resposta.
4. Em caso de sucesso, o frontend utiliza o token retornado e apresenta uma mensagem de confirmação.
5. Se a API estiver indisponível ou ocorrer um erro, o frontend apresenta uma mensagem de falha.

Quando o backend ainda não está pronto, o QA pode criar uma resposta simulada no Mockoon e testar o comportamento do frontend sem depender da API real.

5. Como descobrir as informações necessárias para criar o mock?

A aula apresenta duas alternativas:

Pelo **código-fonte:** identificar a URL da API, o método HTTP, o endpoint, os parâmetros enviados e a estrutura da resposta.

Pelo **Swagger:** consultar a documentação da API para verificar os endpoints, os métodos, os dados de entrada e as respostas previstas.

O instrutor também destaca a importância de validar a documentação. Se o Swagger indicar uma estrutura de resposta diferente daquela retornada pela API, o QA deve comunicar a divergência aos desenvolvedores.

6. O que o QA deve validar?

- Se o frontend envia a requisição correta.
- Se o método HTTP e o endpoint estão corretos.
- Se os campos obrigatórios são enviados.
- Se a aplicação trata respostas de sucesso.
- Se a aplicação apresenta mensagens adequadas para erros.
- Se o frontend funciona mesmo quando o backend real ainda não está disponível.
- Se a estrutura das respostas simuladas corresponde ao contrato documentado da API.

7. Conceitos importantes

Conceito	**Significado**
API	Interface que permite a comunicação entre sistemas ou componentes.
Endpoint	Endereço específico utilizado para acessar uma funcionalidade da API.
Mock	Simulação de um serviço ou componente.
Mockoon	Ferramenta para criar APIs simuladas.
Swagger	Ferramenta de documentação e exploração de APIs.
Token	Dado utilizado, neste exemplo, para representar a autenticação do usuário.
Teste isolado	Teste de um componente sem depender de todos os demais componentes reais.

Resumo final

O principal aprendizado é que o QA não precisa esperar o backend ficar pronto para começar a testar o frontend. Com um mock, é possível simular as respostas da API, verificar os comportamentos esperados e antecipar a identificação de defeitos.

A atividade central do QA é compreender o contrato da API — método, endpoint, dados enviados e respostas esperadas —, configurar as simulações necessárias e validar se o frontend reage corretamente a cada cenário.

---

7. PRINCIPAIS ERROS EM APLICAÇÕES WEB

1. Objetivo da aula

Aprender a identificar os defeitos mais comuns em aplicações web e utilizá-los como referência durante os testes de QA (Quality Assurance).

A principal recomendação do instrutor é procurar esses erros intencionalmente, executar ações que possam revelar falhas e registrar evidências para que o time de desenvolvimento consiga corrigi-las.

2. Os 7 principais erros apresentados

1. Validações incorretas de formulários
O sistema aceita informações inválidas ou aplica máscaras incorretamente.
Exemplo: um cadastro permite salvar um e-mail malformado, como julio@@.
Como testar: insira dados válidos, inválidos, incompletos e em formatos inesperados. Tente salvar e confirme se o sistema impede o armazenamento de informações inválidas.

2. Links e botões quebrados
Links não funcionam, botões não executam a ação esperada ou direcionam para páginas inexistentes, como erro HTTP 404.
Como testar: percorra os links e botões da aplicação. Em sites publicados, ferramentas como o Broken Link Checker podem ajudar a identificar links quebrados automaticamente.

3. Fluxo de navegação interrompido
As informações se perdem ou uma etapa impede a conclusão da jornada do usuário.
Exemplo: o usuário adiciona produtos ao carrinho, mas, ao finalizar a compra, o carrinho aparece vazio.
Como testar: execute o fluxo completo, desde o início até o objetivo final, verificando se os dados são preservados entre as etapas.

4. Erros de cálculo ou processamento
A aplicação apresenta resultados matemáticos ou financeiros incorretos.
Exemplo: cinco produtos de R$ 50,00 deveriam totalizar R$ 250,00, mas o sistema apresenta R$ 249,99.

Como testar: calcule os resultados esperados independentemente e compare-os com os valores apresentados pelo sistema. Verifique subtotais, descontos, impostos e totais.

5. Mensagens de erro inadequadas

Mensagens podem conter erros ortográficos, informações confusas ou textos diferentes dos definidos nos requisitos.

Como testar: provoque erros intencionalmente e verifique a clareza, a ortografia e a conformidade das mensagens com os critérios de aceitação e o conteúdo aprovado pelo produto.

6. Problemas com sessões de usuário

A sessão não é encerrada corretamente após o logout, permitindo acesso indevido a páginas restritas.

Como testar: faça login, encerre a sessão e tente acessar diretamente uma URL protegida. Confirme se a aplicação exige autenticação novamente.

Esse teste também contribui para a verificação da segurança e do controle de acesso.

7. Mensagens de erro não amigáveis

A aplicação exibe erros técnicos que não ajudam o usuário a entender o problema.

Exemplo: aparece Fatal Error em vez de uma orientação clara sobre o campo inválido.

Como testar: provoque situações de erro e verifique se a mensagem explica o problema em linguagem compreensível, sem expor detalhes técnicos internos.

3. Ferramenta demonstrada na aula

O instrutor apresenta o Broken Link Checker, uma ferramenta online que analisa links de sites publicados.

Durante a demonstração:

- No site Terra, foram analisadas 563 páginas sem identificação de links quebrados naquela execução.
- No portal gov.br, foram encontrados links que retornavam erro 404.
- O instrutor mostrou como localizar a página de origem e identificar o link defeituoso para comunicar o problema ao desenvolvedor.

Esses resultados representam apenas as páginas verificadas naquela execução, não uma garantia de que todo o site esteja livre de problemas.

4. Como transformar os achados em registros de QA

Ao identificar um defeito, registre:

Campo	**O que informar**
Título	Descrição curta e objetiva do defeito
Pré-condições	Estado necessário para executar o teste
Passos	Ações realizadas para reproduzir o problema
Resultado esperado	Comportamento correto conforme os requisitos
Resultado obtido	O que aconteceu durante o teste
Evidências	Capturas de tela, vídeos, mensagens ou respostas HTTP
Severidade	Impacto do defeito no funcionamento ou no usuário

Exemplo de registro:

BUG-001 — Carrinho esvazia ao iniciar checkout

Módulo: Carrinho de compras
Passos para reproduzir

1. Acessar a loja.
2. Adicionar um produto ao carrinho.
3. Clicar em “Finalizar compra”.

Resultado esperado: os produtos adicionados continuam disponíveis no checkout.
Resultado obtido: o carrinho aparece vazio.
Evidência: captura de tela do carrinho antes e depois de iniciar o checkout.

5. Checklist prático para seus testes

Checklist de QA
[ ] Validar formulários com dados válidos e inválidos
[ ] Testar todos os links e botões importantes
[ ] Executar os fluxos completos do usuário
[ ] Conferir cálculos, descontos e valores finais
[ ] Validar mensagens de erro e textos da interface
[ ] Verificar logout e proteção de páginas restritas
[ ] Confirmar que erros técnicos não são exibidos ao usuário

Principal aprendizado
Um bom QA não testa apenas o caminho feliz. Ele também verifica entradas inválidas, caminhos alternativos, cálculos, mensagens, continuidade dos dados e controles de acesso.
Esses sete grupos de erros servem como uma referência prática para elaborar casos de teste, encontrar defeitos antes que cheguem aos usuários e comunicar problemas de maneira clara ao time de desenvolvimento.

---

8. SOBRE O CYPRESS

O Cypress é uma ferramenta de automação de testes muito utilizada no mercado de tecnologia, especialmente para automatizar testes de aplicações web. Nesta aula, o foco foi conhecer a ferramenta e aprender a consultar sua documentação antes de começar a escrever os testes.

1. O que é o Cypress?

O Cypress reúne recursos para criar e executar testes automatizados em aplicações web, incluindo:

- **Execução de testes:** executa os cenários automatizados.
- **Interação com o navegador:** permite clicar em botões, preencher campos e validar informações.
- **Visualização dos testes:** permite acompanhar a execução e identificar falhas.
- **Bibliotecas e comandos:** oferece funções prontas para escrever os scripts de teste.

2. Documentação oficial

Cypress.io
Documentação oficial

Principais áreas da documentação

Área	Para que serve
Cypress App	Conhecer a aplicação utilizada para executar e acompanhar os testes.
End-to-End Testing	Automatizar jornadas completas do usuário na aplicação.
Component Testing	Testar componentes de forma isolada.
Core Concepts	Entender os principais conceitos do Cypress.
Guides	Consultar guias de uso e boas práticas.
Continuous Integration	Aprender a executar testes em pipelines de integração contínua.
API	Consultar os comandos e suas formas de utilização.
Custom Commands	Aprender a criar comandos personalizados para reutilizar código.

3. Primeiro comando: cy.visit()

O comando cy.visit() permite acessar uma URL durante a execução do teste.

Exemplo:

```
describe('Meu primeiro teste', () => {
  it('deve acessar uma página web', () => {
    cy.visit('https://example.cypress.io')
  })
})
```

Entenda cada parte:

- describe() agrupa testes relacionados.
- it() define um cenário de teste.
- cy.visit() acessa a página indicada.

Nesse exemplo, o teste abre o site de demonstração do Cypress. Ele ainda não valida se algum elemento está correto: apenas acessa a página.

A documentação do comando está disponível em Cypress — cy.visit().

4. O que são Custom Commands?

Os **Custom Commands** são comandos personalizados que permitem encapsular ações reutilizáveis.

Por exemplo, se vários testes precisam realizar o login, podemos criar um comando próprio para centralizar essa ação, evitando repetir o mesmo código em todos os cenários.

Isso ajuda a manter os scripts organizados e facilita a manutenção da automação.

Documentação: Custom Commands.

5. O que você precisa aprender como QA

Para seu objetivo profissional em Quality Assurance, sugiro seguir esta sequência:

1. **Fundamentos de JavaScript:** variáveis, funções, condições, arrays e objetos.
2. **Instalação e configuração do Cypress:** preparar o ambiente e criar o projeto de testes.
3. **Comandos básicos:** cy.visit(), cy.get(), cy.contains() e cy.click().
4. **Validações:** utilizar should() e outras asserções para verificar os resultados esperados.
5. **Cenários positivos e negativos:** validar tanto o comportamento correto quanto as situações de erro.
6. **Organização dos testes:** trabalhar com seletores, fixtures, comandos personalizados e boas práticas.
7. **Projeto prático de QA:** criar testes automatizados, registrar defeitos encontrados e documentar os resultados.

---

9. CRIANDO O REPOSITÓRIO DE TESTES AUTOMATIZADOS WEB

1. O que é um repositório de testes?
É um espaço onde você armazena e organiza os arquivos relacionados à automação de testes de uma aplicação.

Neste curso, existem dois projetos relacionados:

- Banco Web: a aplicação web que será testada.
- Banco Web Tests: o projeto que armazenará os scripts de testes automatizados.

A aplicação Banco Web se comunica com a Banco API. Nosso trabalho como QA será verificar se os comportamentos da aplicação funcionam conforme o esperado.

2. Etapas realizadas na aula

1. Criar o repositório no GitHub

Nome: banco-web-tests
Descrição: testes automatizados do aplicativo web do banco.
O nome pode variar conforme o que foi usado pelo instrutor, mas é recomendável manter o mesmo nome no seu projeto.

2. Selecionar o .gitignore do Node
O GitHub oferece um modelo que ignora arquivos e pastas que não devem ser versionados, como dependências instaladas e outros arquivos gerados localmente.
Isso é útil porque o Cypress será instalado em um projeto Node.js.

3. Clonar o repositório
No GitHub, copiar a URL HTTPS e executar no terminal: **git clone URL_DO_REPOSITORIO**
Substitua URL_DO_REPOSITORIO pela URL HTTPS copiada do seu repositório.

4. Abrir o projeto no VS Code
No VS Code, acessar File > Open Folder e selecionar a pasta clonada.

3. Conceitos importantes para QA

Conceito	**Significado**
GitHub	Plataforma para hospedar e compartilhar repositórios Git.
Git	Sistema de controle de versão do código.
Repositório	Local onde os arquivos e seu histórico de alterações são armazenados.
git clone	Copia um repositório remoto para o computador.
.gitignore	Define arquivos e pastas que o Git deve ignorar.
VS Code	Editor utilizado para trabalhar nos arquivos do projeto.

4. Como conferir se deu certo?
No terminal do VS Code, dentro da pasta do projeto, execute: **git status**
Se aparecer algo semelhante a:
On branch main
nothing to commit, working tree clean
significa que o repositório foi reconhecido pelo Git e que não há alterações pendentes.
O nome da branch pode ser diferente, dependendo da configuração do GitHub.
Você também pode executar: **git remote -v**
Esse comando mostra o endereço do repositório remoto configurado para o projeto.

5. O que vem na próxima aula?

O próximo passo será preparar o projeto para a automação:

- Inicializar ou configurar o projeto Node.js, se necessário.
- Instalar o Cypress.
- Abrir a aplicação do Cypress.

- Criar a estrutura inicial dos testes.
- Executar o primeiro teste automatizado.

Dica profissional: em projetos reais, separar os testes automatizados do código da aplicação pode facilitar a manutenção, a execução independente da suíte de testes e a integração com pipelines de CI/CD.

Não precisa instalar nada além do que o instrutor orientar nesta etapa. Primeiro, garanta que o repositório foi clonado e aberto corretamente no VS Code.

---

10. INSTALANDO E CONFIGURANDO O CYPRESS

1. O que você precisa entender

O Cypress é uma ferramenta de automação de testes, muito utilizada para testar aplicações web. Com ele, você pode automatizar ações como:

- Acessar uma página de login.
- Preencher campos de usuário e senha.
- Clicar em botões.
- Validar mensagens de sucesso ou erro.
- Verificar se o comportamento da aplicação corresponde ao esperado.

Nesta aula, você configurou o ambiente para começar a escrever testes automatizados do tipo End-to-End (E2E).

2. Passo a passo dos comandos

Passo 1 — Abrir o terminal
No VS Code, abra o terminal pelo menu Terminal → New Terminal. O professor escolheu o CMD em vez do PowerShell, mas isso não é uma exigência geral do Cypress.

Passo 2 — Inicializar o projeto Node.js
Execute: **npm init -y**
Isso cria o arquivo package.json, que registra informações e configurações do projeto, incluindo suas dependências.

Passo 3 — Instalar o Cypress
Para reproduzir a versão demonstrada na aula: **npm install --save-dev cypress**
O parâmetro --save-dev registra o Cypress como dependência de desenvolvimento. O diretório node_modules armazena os pacotes instalados e o package-lock.json registra a árvore de dependências e suas versões.

Passo 4 — Abrir a interface do Cypress
Execute: **npx cypress open**
Esse comando abre a interface gráfica para configurar e executar testes.

Passo 5 — Configurar os testes E2E

Na interface do Cypress:

1. Selecione End-to-End Testing.
2. Conclua a configuração inicial dos arquivos.
3. Escolha um navegador disponível, como Electron ou Chrome.
4. Crie uma nova spec chamada login.cy.js.
5. Execute o teste de exemplo.

3. Entendendo a estrutura de pastas

Depois da configuração, a estrutura pode ficar semelhante a esta:

seu-projeto/
package.json
package-lock.json
node_modules/
cypress/
e2e/
login.cy.js
fixtures/
support/
cypress.config.js

Os arquivos e diretórios têm funções diferentes:

Arquivo ou pasta	**Para que serve**
package.json	Configura o projeto e registra dependências.
package-lock.json	Registra versões específicas e a árvore de dependências.
node_modules	Armazena os pacotes instalados.
cypress.config.js	Define configurações do Cypress.
e2e	Armazena os testes End-to-End.
fixtures	Guarda dados de teste, como informações de usuários fictícios.
support	Guarda configurações e comandos de apoio aos testes.

Os nomes e arquivos exatos podem variar conforme a versão e as opções escolhidas durante a configuração.

4. Seu primeiro teste automatizado

O Cypress utiliza a estrutura de testes do Mocha. O exemplo abaixo mostra a lógica ensinada pelo professor:

```
describe('Teste de login', () => {
  it('deve acessar a página de exemplo', () => {
```

```
  cy.visit('https://example.cypress.io')
 })
})
```

Entenda cada parte:

- describe() agrupa os testes de um contexto, neste caso, o login.
- it() define um caso de teste específico.
- cy.visit() manda o Cypress acessar uma URL.
- () => {} representa uma função arrow, usada para definir o que será executado.

Atenção: esse exemplo apenas acessa uma página. Ele ainda não testa um login, nem valida se alguma funcionalidade está correta.

5. O que você deve aprender como QA

É importante distinguir três coisas:

- Instalar: preparar as ferramentas necessárias.
- Configurar: definir como os testes serão organizados e executados.
- Automatizar: escrever instruções e verificações que comprovem o comportamento esperado da aplicação.

Na prática profissional, executar um teste sem erros não significa automaticamente que o sistema está correto. Você precisa definir o resultado esperado e verificar se ele realmente aconteceu.

---

11. CRIANDO O MEU PRIMEIRO SCRIPT DE TESTE AUTOMATIZADO COM CYPRESS

1. Objetivo da aula

Automatizar o login de um sistema bancário usando dados válidos.

O script deverá:

1. Acessar o sistema.
2. Localizar o campo de usuário e clicar nele.
3. Digitar o nome de usuário.
4. Localizar o campo de senha e digitar a senha.
5. Clicar no botão de entrar.
6. Na próxima aula, adicionar uma asserção para validar se o login funcionou.

2. Código completo do teste

No arquivo cypress/e2e/login.cy.js, coloque:

```
describe('Login', () => {
  it('Login com dados validos deve permitir entrada no sistema', () => {
    cy.visit('http://localhost:4000')
    cy.get('#username').click().type('julio.lima')
    cy.get('#senha').click().type('123456')
```

```
    cy.get('#login-section > .btn').click()
  });
});
```

Seletores CSS: o símbolo # identifica um elemento pelo seu ID. Por exemplo, cy.get('#username') localiza o elemento cujo HTML contém id="username".

Encadeamento: permite executar ações em sequência sobre um elemento, evitando repetir a localização desnecessariamente.

cy.get('#username').click().type('Julio.Lima')

4. Por que precisamos de uma asserção?
O script executa as ações de login, mas ainda não verifica explicitamente se o acesso foi realizado com sucesso.

Um teste completo precisa contemplar:

- **Preparação:** acessar o sistema e estabelecer as condições necessárias.
- **Ações:** preencher os campos e clicar em entrar.
- **Asserção:** verificar se o resultado corresponde ao esperado.

---

12. ADICIONANDO ASSERÇÕES

Assertions

1. O que é uma asserção?
Uma asserção é uma verificação que compara o resultado atual da aplicação com o resultado esperado.
No teste de login, o objetivo é:

Login com dados válidos deve permitir a entrada no sistema.
Para verificar se o login funcionou, podemos conferir se o título **Realizar transferência** está visível após a autenticação.

2. Código completo do teste

```
  cy.contains('h4', 'Realizar transferência') should('be.visible')
```

Como funciona a asserção:

- cy.contains('h4', 'Realizar transferência'): procura um elemento h4 que contenha o texto informado.
- .should('be.visible'): verifica se o elemento está visível na página.

Se o título estiver visível, a verificação passa. Se não estiver visível dentro do tempo de espera do Cypress, a asserção falha.

3. Por que as asserções são importantes?

Elas permitem identificar defeitos que impedem a aplicação de apresentar o resultado esperado.

Por exemplo, se um desenvolvedor alterar incorretamente o código de autenticação, o login poderá falhar. Nesse caso, o título de transferência não ficará visível e o teste identificará o problema.

Esse processo ajuda a detectar regressões, ou seja, situações em que uma alteração prejudica uma funcionalidade que deveria continuar funcionando.

4. Outros exemplos de asserções

O método should() aceita diferentes verificações, chamadas *chainers*.

Asserção	**O que verifica**
.should('be.visible')	O elemento está visível.
.should('exist')	O elemento existe no DOM.
.should('not.exist')	O elemento não existe no DOM.
.should('be.enabled')	O elemento está habilitado.
.should('have.text', 'Texto esperado')	O texto do elemento corresponde exatamente ao esperado.
.should('contain.text', 'Texto')	O elemento contém o texto informado.

A documentação oficial apresenta outras opções: https://docs.cypress.io/api/commands/should

5. Conceito importante: teste como contrato

O teste documenta um comportamento esperado da aplicação. Quando uma alteração faz o teste falhar, é necessário investigar a causa.

A falha pode indicar um defeito no código, mas também pode decorrer de uma mudança legítima na aplicação ou de um problema no próprio teste. Por isso, o resultado deve ser analisado antes de concluir que existe um bug.

Em projetos profissionais, os testes automatizados podem ser executados em pipelines de CI/CD, ajudando a identificar problemas antes que as alterações sejam integradas ou publicadas.

---

13. ADICIONANDO UM SEGUNDO TESTE A SUITE

1. Objetivo da aula
Adicionar um segundo teste automatizado para verificar o comportamento do sistema quando o usuário informa uma senha incorreta.

Ao final, a suíte terá dois testes:

1. **Login válido:** verifica se o usuário consegue entrar no sistema.
2. **Login inválido:** verifica se o sistema apresenta a mensagem Erro no login, tente novamente.

2. Código completo dos testes
Arquivo: cypress/e2e/login.cy.js

```
 it('Login com dados inválidos deve apresentar mensagem de erro', () => {
   cy.visit('http://localhost:4000')

   cy.get('#username').click().type('Julio.Lima')
   cy.get('#password').click().type('65432')
   cy.contains('button', 'Entrar').click()
   cy.get('.toast').should('be.visible'). and('have.text', 'Erro no login, tente novamente')
 })
```

Atenção: confirme se os seletores, as credenciais, o título e a mensagem correspondem ao projeto utilizado na aula. A classe .toast e o texto exato precisam ser os mesmos da aplicação. Se a mensagem incluir espaços ou quebras de linha, a asserção de texto exato poderá precisar de ajuste.

3. Entendendo o segundo teste
O segundo it() executa estas etapas:

1. Abre a página de login.
2. Preenche o usuário válido.
3. Preenche uma senha incorreta.
4. Clica no botão **Entrar**.
5. Verifica se a mensagem de erro aparece.
6. Verifica se a mensagem contém o texto esperado, exatamente.

Comandos importantes

Comando	**Finalidade**
cy.visit()	Abre a página da aplicação.
cy.get('.toast')	Localiza o elemento pela classe CSS toast.
.should('be.visible')	Verifica se o elemento está visível.
.and('have.text', '...')	Verifica se o texto completo corresponde ao esperado.
cy.contains('button', 'Entrar')	Localiza um botão que contém o texto Entrar.

Diferença entre have.text e contain.text

- **have.text:** verifica se o texto completo corresponde ao esperado.
- **contain.text:** verifica se o texto contém um trecho específico.

Exemplo: **cy.get('.toast').should('contain.text', 'Erro no login')**

Essa segunda opção permite validar apenas uma parte da mensagem.

4. O que é uma suíte de testes?

Uma **suíte de testes** é um conjunto de testes automatizados relacionados.
Neste exemplo, o describe('Login', ...) agrupa os dois cenários de login, enquanto cada it() representa um teste independente.

5. Executar apenas um teste

Durante o desenvolvimento, é possível executar somente um teste usando .only:

```
it.only('Login com dados inválidos deve apresentar mensagem de erro', () => {
  // Código do teste
})
```

Para executar novamente toda a suíte, remova o .only.

Importante: não deixe it.only ou describe.only no código enviado ao repositório, pois os demais testes podem deixar de ser executados.

6. Usando cy.contains() para melhorar a leitura
Em vez de localizar o botão por um seletor técnico, como: **cy.get('#login-selection-btn').click()**
Podemos utilizar: **cy.contains('button', 'Entrar').click()**

A segunda forma deixa mais claro o objetivo do comando: clicar no botão que contém o texto Entrar.

O cy.contains() recebe o tipo do elemento e o texto que ele deve conter. O texto precisa corresponder ao conteúdo da interface, respeitando diferenças relevantes de escrita.

7. Depurando o teste no Cypress
O Cypress permite examinar os estados da aplicação antes e depois de cada comando.

Na janela do Cypress:

1. Execute o teste.
2. Selecione o comando desejado no histórico.
3. Examine o estado anterior (*before*) e o posterior (*after*).
4. Inspecione o elemento para identificar seus seletores CSS e seu conteúdo.

Esse recurso ajuda a descobrir por que um teste falhou e quais elementos estavam presentes na tela.

8. Versionando a alteração com Git
O fluxo apresentado na aula é criar uma branch para o novo teste, implementar a alteração e enviá-la ao GitHub para revisão.

Os comandos abaixo são exemplos. Use o nome real da sua branch e confirme se o repositório está atualizado antes de começar.

```
git checkout main
git pull
git checkout -b adicionando-novo-teste-login-falho
```
Depois de implementar e testar o código:
```
git status
git add .
git status
git commit -m "Adiciona teste para login com credenciais inválidas"
git push -u origin adicionando-novo-teste-login-falho
```

Em seguida, abra um **Pull Request** no GitHub para revisar e integrar a alteração à branch principal.

Se a branch main tiver novas alterações depois do seu trabalho, atualize sua branch conforme o fluxo adotado pelo projeto antes de concluir a revisão.

9. Checklist de aprendizado

- Criar um segundo it() dentro do mesmo describe().
- Diferenciar o cenário de login válido do inválido.
- Validar a visibilidade de uma mensagem com be.visible.
- Validar o texto completo com have.text.
- Entender a diferença entre cy.get() e cy.contains().
- Executar apenas um teste com it.only e depois removê-lo.
- Executar os dois testes juntos.
- Criar uma branch, fazer commit e enviar a alteração ao GitHub.

Principal aprendizado: um teste automatizado não deve apenas executar ações na interface. Ele também precisa verificar se a aplicação apresentou o resultado esperado para cada cenário.

---

14. REDUZINDO DUPLICAÇÃO DE CÓDIGO COM HOOKS

Bundled Libraries

1. Objetivo

Utilizar hooks do Mocha para executar ações comuns aos testes sem repetir código.

2. Código com beforeEach()

Arquivo: cypress/e2e/login.cy.js

```
describe('Login', () => {
  beforeEach(() => {
    cy.visit('http://localhost:4000')
  })

  it('Login válido deve permitir entrada no sistema', () => {
    cy.get('#username').type('Julio.Lima')
    cy.get('#password').type('23456')
    cy.contains('button', 'Entrar').click()

    cy.contains('h4', 'Realizar transferência')
      .should('be.visible')
  })

  it('Login inválido deve apresentar mensagem de erro', () => {
    cy.get('#username').type('Julio.Lima')
    cy.get('#password').type('65432')
    cy.contains('button', 'Entrar').click()

    cy.get('.toast')
      .should('be.visible')
      .and('have.text', 'Erro no login, tente novamente')
  })
})
```

Confirme os seletores e os textos conforme a aplicação utilizada na aula.

3. Hooks do Mocha

Hook	Execução
before()	Uma vez antes de todos os testes do bloco.
beforeEach()	Antes de cada teste.
afterEach()	Depois de cada teste.
after()	Uma vez depois de todos os testes do bloco.

Neste exemplo: beforeEach() abre a página antes de cada it(). Assim, não precisamos repetir cy.visit() nos dois testes.

4. Versionamento com Git

git checkout main
git pull
git checkout -b adicionando-before-each
Após alterar o código e verificar os testes:
git status
git add .
git commit -m "Adiciona beforeEach para acesso à página de login"
git push -u origin adicionando-before-each
Depois, abra um Pull Request no GitHub para revisão. Após o merge, atualize a branch principal:
git checkout main
git pull

5. Prática

- Identificar o código repetido.
- Movê-lo para beforeEach().
- Executar os dois testes e confirmar que passaram.
- Versionar a alteração no GitHub.

Conceito-chave: hooks organizam ações que precisam ocorrer em momentos específicos da execução dos testes.

---

15. EXECUÇÃO DOS TESTES VIA LINHA DE COMANDO

Command Line

1. Modos de execução

- **Headless:** executa os testes sem exibir o navegador. É comum em pipelines de CI/CD.
- **Headed:** executa os testes com o navegador visível.

2. Comandos do Cypress

Comando	Finalidade
npx cypress open	Abre a interface gráfica do Cypress.
npx cypress run	Executa os testes em modo headless.
npx cypress run --headed	Executa os testes com o navegador visível.
npx cypress run --browser chrome	Executa os testes no Chrome, se disponível.
npx cypress run --spec "cypress/e2e/login.cy.js"	Executa somente o arquivo especificado.

3. Configurando scripts no package.json

Adicione os atalhos abaixo à seção scripts do arquivo, preservando as configurações existentes:

```
{
  "scripts": {
    "test": "cypress run",
    "cy:headed": "cypress run --headed",
    "cy:open": "cypress open"
  }
}
```

Executando os scripts

Comando	Resultado
npm test	Executa os testes em modo headless.
npm run cy:headed	Executa os testes com o navegador visível.
npm run cy:open	Abre a interface gráfica do Cypress.

O npm test executa diretamente o script test. Para os outros scripts, utiliza-se npm run seguido do nome configurado.

4. Aplicação em CI/CD

Em pipelines de integração e entrega contínuas, o Cypress pode executar testes automaticamente para identificar falhas antes da publicação de uma versão.

5. Prática

- Executar npx cypress run e npx cypress run --headed.
- Configurar os scripts no package.json.
- Executar os três scripts usando npm.
- Verificar os resultados dos testes.

---

16. GRAVANDO VIDEOS DAS EXECUÇÕES E SCREENSHOTS DAS FALHAS

1. Configuração no cypress.config.js
Para gravar vídeos das execuções, habilite video: true na configuração principal:

```
const { defineConfig } = require("cypress");

module.exports = defineConfig({
  allowCypressEnv: false,
  video: true,

  e2e: {
    screenshotOnRunFailure: true,

    setupNodeEvents(on, config) {
      // implement node event listeners here
    },
  },
});
```

- **video:** true: grava um vídeo da execução dos testes via linha de comando.
- **screenshotOnRunFailure:** true: tira screenshots quando os testes falham durante cypress run.

2. Onde os arquivos são salvos?

Arquivo	**Caminho padrão**
Vídeos	cypress/videos/
Screenshots	cypress/screenshots/

O Cypress pode substituir os vídeos anteriores quando uma nova execução gera arquivos com os mesmos nomes. Os arquivos ocupam espaço, por isso é importante avaliar se vale a pena mantê-los em todas as execuções.

3. Como funciona o screenshot de falha?

Imagine que o teste espera esta mensagem:

```
cy.get('.toast')
  .should('be.visible')
  .and('have.text', 'Erro no login, tente novamente')
```

Mas a aplicação apresenta Sucesso.

O Cypress identifica a falha na asserção e, com a captura habilitada, salva uma imagem para ajudar a investigar o problema. O vídeo também permite rever a execução.

Importante: não é necessário provocar erros na aplicação para configurar esse recurso. O exemplo da aula foi apenas uma demonstração.

4. Como tirar screenshots manualmente?
Para capturar uma imagem em um ponto específico do teste:
cy.screenshot('login-realizado-com-sucesso')

Essa instrução pode ser colocada depois da validação do login. Diferentemente da captura automática por falha, ela permite registrar também uma execução bem-sucedida.

5. Como equilibrar velocidade e evidências?
Para o seu projeto, recomendo:

- Durante o aprendizado: mantenha video: true para rever as execuções.
- Durante a investigação de falhas: mantenha screenshotOnRunFailure: true.
- Nas execuções rotineiras: se quiser economizar tempo e espaço, remova a captura manual quando não precisar dela. Você pode manter o vídeo habilitado.

A gravação de vídeos pode aumentar o tempo de execução, mas o ganho ao desabilitá-la varia conforme o projeto e o ambiente.

Resumo: vídeo registra a execução; screenshot registra uma imagem de um momento específico; a captura automática de falhas ajuda a investigar testes que não passaram.

---

17. USANDO FIXTURES

Fixture

1. O que são Fixtures?

Fixtures permitem armazenar dados de teste em arquivos externos, separando os dados do código dos testes. Isso facilita a reutilização e a manutenção: se uma credencial mudar, basta alterar o arquivo correspondente.

Por padrão, ficam em cypress/fixtures/.

2. Criando o arquivo de dados

Crie cypress/fixtures/credenciais.json:

```
{
  "valida": {
    "usuario": "Julio.Lima",
    "senha": "23456"
  },
  "invalida": {
```

```
    "usuario": "Julio.Lima",
    "senha": "65432"
  }
}
```

O arquivo contém dois objetos, valida e invalida, cada um com as propriedades usuario e senha.

3. Utilizando a Fixture nos testes

No arquivo cypress/e2e/login.cy.js:

```
describe('Login', () => {
  beforeEach(() => {
    cy.visit('http://localhost:4000')
  })

  it('Login válido deve permitir entrada no sistema', () => {
    cy.fixture('credenciais').then((credenciais) => {
      cy.get('#username').type(credenciais.valida.usuario)
      cy.get('#password').type(credenciais.valida.senha)
      cy.contains('button', 'Entrar').click()

      cy.contains('h4', 'Realizar transferência')
        .should('be.visible')
    })
  })

  it('Login inválido deve apresentar mensagem de erro', () => {
    cy.fixture('credenciais').then((credenciais) => {
      cy.get('#username').type(credenciais.invalida.usuario)
      cy.get('#password').type(credenciais.invalida.senha)
      cy.contains('button', 'Entrar').click()

      cy.get('.toast')
        .should('be.visible')
        .and('have.text', 'Erro no login, tente novamente')
    })
  })
})
```

Como funciona:

- cy.fixture('credenciais'): carrega o arquivo sem precisar informar .json.
- .then((credenciais) => { ... }): disponibiliza os dados carregados.
- credenciais.valida.usuario: acessa o usuário válido.
- credenciais.invalida.senha: acessa a senha inválida.

4. Ignorando screenshots no Git

Adicione ao arquivo .gitignore:
cypress/screenshots/

Isso evita que screenshots ainda não rastreados sejam adicionados ao Git. Se já estiverem sendo rastreados, será necessário removê-los do índice para aplicar a regra.

5. Versionando as alterações
git status
git add .
git status
git commit -m "Adiciona fixtures de credenciais"
git push -u origin adicionando-fixtures

Confira o git status antes do commit e ajuste o nome da branch, se necessário. Depois, abra um Pull Request para revisão e merge.

Após o merge, atualize a branch principal:
git checkout main
git pull

Documentação oficial
https://docs.cypress.io/api/commands/fixture

---

18. UTILIZANDO VARIÁVEIS DE AMBIENTE

1. O que são variáveis de ambiente?

Variáveis de ambiente permitem alterar informações utilizadas durante a execução dos testes sem precisar modificar o código dos scripts.

Na automação de testes, é comum executar os mesmos testes em diferentes ambientes, como local, QA e produção. Como cada ambiente possui um endereço diferente, as variáveis de ambiente permitem configurar qual aplicação será acessada em cada execução.

Essa abordagem evita alterações desnecessárias no código, novos commits e pull requests apenas para modificar endereços.

2. Configurando variáveis de ambiente no package.json

No arquivo package.json, podemos definir comandos diferentes para executar os testes em cada ambiente.

Exemplo:
"scripts": {
  "test": "cypress run --env URL=http://localhost:4000",

```
  "test-qa": "cypress run --env URL=https://qa.exemplo.com",
  "test-prod": "cypress run --env URL=https://exemplo.com"
}
```

Os endereços de QA e produção são exemplos e devem ser substituídos pelos endereços reais dos ambientes.

O parâmetro --env permite enviar variáveis de ambiente para o Cypress. Nesse exemplo, a variável se chama URL e recebe um endereço diferente conforme o comando executado.

Podemos declarar várias variáveis de ambiente, separando-as por vírgulas.

3. Utilizando a variável no script de teste

No arquivo login.cy.js, substituímos o endereço fixo utilizado no comando cy.visit() pela variável de ambiente.
Antes: **cy.visit('http://localhost:4000');**
Depois: **cy.visit(Cypress.env('URL'));**

Dessa forma, o teste utiliza automaticamente o endereço recebido durante a execução. Não é necessário alterar o script sempre que quisermos acessar outro ambiente.

4. Executando os testes em diferentes ambientes

Os comandos definidos no package.json permitem selecionar o ambiente desejado:

- npm test: executa os testes no ambiente local, utilizando http://localhost:4000.
- npm run test-qa: executa os testes no ambiente de QA.
- npm run test-prod: executa os testes no ambiente de produção.

Quando o comando é executado, o Cypress utiliza o valor atribuído à variável URL para acessar a aplicação.

Se o ambiente estiver disponível e apresentar as condições esperadas pelo teste, a execução poderá ser concluída com sucesso. Caso contrário, o teste poderá falhar.

5. Verificando os resultados com screenshots

Os screenshots ajudam a visualizar a página acessada durante a execução dos testes.

Ao utilizar variáveis de ambiente, podemos verificar se o Cypress realmente abriu o endereço configurado.

Por exemplo, ao executar um teste contra outro ambiente, o screenshot pode mostrar a página correspondente a esse endereço.

As capturas também auxiliam na investigação de falhas e na identificação de problemas durante a execução.

6. Vantagens das variáveis de ambiente

- Permitem executar os mesmos testes em diferentes ambientes.
- Evitam alterar o código dos testes para trocar endereços.
- Facilitam a manutenção dos scripts.
- Reduzem alterações desnecessárias no controle de versão.
- Permitem configurar diferentes valores para cada execução.

Atenção: testes em produção devem ser realizados somente com autorização e com cuidado para não modificar dados reais.

Conclusão

As variáveis de ambiente tornam os testes automatizados mais flexíveis e reutilizáveis. No Cypress, podemos definir os valores no package.json utilizando --env e recuperá-los no script por meio de Cypress.env().

Assim, o mesmo teste de login pode ser executado em diferentes ambientes sem precisar modificar seu código.

---

19. DEFININDO A CONFIGURAÇÃO BASEURL

1. O que é baseUrl?

O baseUrl define o endereço-base da aplicação que será testada. Ele evita repetir a URL completa em cada teste.

Exemplo de endereço-base: http://localhost:4000

Páginas da aplicação:

- http://localhost:4000/login
- http://localhost:4000/usuarios
- http://localhost:4000/transferencias

2. Como configurar

No arquivo cypress.config.js, adicione baseUrl dentro de e2e:

```
const { defineConfig } = require('cypress');

module.exports = defineConfig({
  e2e: {
    baseUrl: 'http://localhost:4000',
  },
});
```

Se o arquivo já tiver outras configurações, mantenha-as e adicione apenas a propriedade baseUrl dentro de e2e.

3. Como usar nos testes
Sem baseUrl, é necessário informar o endereço completo:
cy.visit('http://localhost:4000/login');

Com baseUrl, basta informar o caminho:
cy.visit('/login');

O Cypress acessará http://localhost:4000/login.
A mesma configuração pode ser usada em requisições HTTP:
cy.request('/usuarios');

O endpoint /usuarios precisa existir na aplicação ou API testada.

4. Diferença entre baseUrl e variáveis de ambiente

- **baseUrl:** define o endereço-base da aplicação e é utilizado automaticamente por comandos como cy.visit() e cy.request() quando recebem caminhos relativos.
- **Variáveis de ambiente:** armazenam valores que podem variar conforme a execução, como tokens e parâmetros de teste.

5. Resumo para QA

- **Quando usar:** quando a parte inicial da URL se repete nos testes.
- **Onde configurar:** cypress.config.js, dentro de e2e.
- **Benefício:** reduz a repetição e facilita a manutenção dos testes.
- **Exemplo:** cy.visit('/login').

---

20. NAVEGANDO NA ESTRUTURA HIERÁRQUICA DE ELEMENTOS SEM ID ÚNICO
Select

Cypress — Navegando na Estrutura Hierárquica de Elementos sem ID Único
1. Objetivo da aula
Aprender a localizar e interagir com elementos HTML quando eles não possuem um id único, utilizando a estrutura hierárquica da página e os comandos do Cypress.

2. Estrutura básica de um teste

- describe(): agrupa testes relacionados.
- it(): define um caso de teste.
- beforeEach(): executa uma preparação antes de cada teste.
- cy.visit('/'): acessa a página inicial definida em baseUrl.
- cy.get(): localiza elementos usando seletores CSS.
- cy.contains(): localiza elementos pelo texto.
- cy.click(): realiza um clique.
- cy.type(): digita um valor.
- cy.should(): verifica uma condição.

Exemplo:

```
describe('Transferência bancária', () => {
  beforeEach(() => {
    cy.visit('/');
  });

  it('deve realizar uma transferência', () => {
    // Etapas do teste
  });
});
```

3. Localizando elementos pela hierarquia HTML

Quando um elemento não possui um identificador exclusivo, podemos localizar um elemento relacionado e navegar pela estrutura HTML.

Exemplo:

```
cy.get('label[for="conta-origem"]')
  .parent()
  .as('campoContaOrigem');
```

Entenda os comandos:

- cy.get(): localiza o elemento pelo seletor CSS.
- label[for="conta-origem"]: procura um elemento label cujo atributo for tenha esse valor.
- .parent(): seleciona o elemento pai direto.
- .as('campoContaOrigem'): cria um alias para reutilizar a seleção.

Para reutilizar o alias:

```
cy.get('@campoContaOrigem').click();
```

Atenção: parent() só seleciona o pai direto. É necessário conferir o HTML para garantir que o elemento encontrado seja o correto.

4. Selecionando uma opção

Em um componente de seleção personalizado, podemos localizar a área do campo e clicar na opção desejada:

```
cy.get('label[for="conta-origem"]')
  .parent()
  .click();

cy.contains('Priscila').click();
```

O primeiro comando abre o campo; o segundo procura o texto Priscila e clica no elemento correspondente.

O seletor precisa corresponder à estrutura real da página. Se houver mais de um elemento com esse texto, restrinja a busca ao componente correto.

Diferença entre select e seletores personalizados

Para um elemento HTML nativo <select>, o Cypress oferece o comando .select():
cy.get('select[name="conta-origem"]').select('Priscila');

Esse exemplo depende de o elemento ser realmente um <select> e de Priscila corresponder a um valor ou texto de opção válido.

Em componentes personalizados, normalmente é necessário abrir o campo e clicar na opção exibida.

5. Preenchendo um campo
Exemplo de preenchimento do valor da transferência:

```
cy.get('#valor')
  .clear()
  .type('11');
```

- #valor: localiza o elemento pelo id.
- .clear(): limpa o conteúdo atual.
- .type('11'): digita o valor 11.

6. Clicando no botão e validando o resultado
Para clicar no botão pelo texto:
cy.contains('button', 'Transferir').click();

Para verificar se a mensagem de sucesso foi exibida:
cy.contains('Transferência realizada').should('be.visible');

A validação é importante porque o teste não deve apenas executar ações: ele também precisa verificar o resultado esperado.

7. Boas práticas

- Prefira seletores estáveis e exclusivos.
- Quando possível, solicite identificadores específicos para automação, como data-cy.
- Evite depender de posições ou estruturas HTML que mudam com frequência.
- Confira o HTML antes de utilizar parent().
- Utilize aliases quando precisar reutilizar seleções.
- Faça asserções para confirmar os resultados.
- Garanta que os testes possam ser executados novamente sem depender de dados deixados por execuções anteriores.

Exemplo de seletor recomendado, quando o atributo existir no HTML:
cy.get('[data-cy="valor-transferencia"]').clear().type('11');

8. Referências

Documentação oficial do Cypress

- Documentação geral: https://docs.cypress.io/
- Organização de testes: https://docs.cypress.io/app/core-concepts/writing-and-organizing-tests
- Comando parent(): https://docs.cypress.io/api/commands/parent
- Comando contains(): https://docs.cypress.io/api/commands/contains
- Comando as(): https://docs.cypress.io/api/commands/as
- Boas práticas: https://docs.cypress.io/app/core-concepts/best-practices

W3Schools — HTML

- Elemento <select>: https://www.w3schools.com/tags/tag_select.asp
- HTML DOM: https://www.w3schools.com/js/js_htmldom.asp

9. O que preciso lembrar

1. cy.get() localiza elementos por seletores.
2. parent() permite navegar até o elemento pai direto.
3. as() cria um alias para reutilizar uma seleção.
4. contains() localiza elementos pelo texto.
5. .select() é usado em elementos HTML nativos <select>.
6. .click() executa cliques e .type() preenche campos.
7. should() verifica se o resultado esperado ocorreu.

Conclusão: para automatizar elementos sem id único, preciso entender a estrutura HTML, escolher seletores confiáveis e validar o resultado das ações.

---

21. COMANDOS CUSTOMIZADOS (CUSTOM COMMANDS) E O REAPROVEITAMENTO DE CÓDIGO

1. O que são Custom Commands?

Custom Commands são comandos personalizados criados para reaproveitar código nos testes automatizados.

Em vez de repetir várias linhas para realizar uma mesma ação, criamos um comando que pode ser chamado sempre que necessário.

Exemplos de situações que podem virar comandos customizados:

- Fazer login com credenciais válidas.
- Fazer login com credenciais inválidas.
- Selecionar uma opção em uma combo box.
- Verificar mensagens de Toast.
- Realizar uma transferência bancária.

2. Como organizar os arquivos
Na pasta cypress/support/, criar uma pasta chamada commands com os seguintes arquivos:

```
cypress/
└── support/
    ├── commands.js
    └── commands/
        ├── common.js
        ├── login.js
        └── transferencias.js
```

- common.js: comandos reutilizáveis em diferentes funcionalidades.
- login.js: comandos específicos da funcionalidade de login.
- transferencias.js: comandos específicos de transferências.
- commands.js: arquivo original do Cypress que pode importar os arquivos personalizados.

No arquivo cypress/support/commands.js, importar os módulos:

```
import './commands/common';
import './commands/login';
import './commands/transferencias';
```

Importante: os caminhos devem corresponder aos nomes reais dos arquivos e pastas do projeto. Essa estrutura utiliza módulos JavaScript com import.

3. Criando comandos customizados
O comando Cypress.Commands.add() registra um comando personalizado.

Estrutura básica:

```
Cypress.Commands.add('nomeDoComando', () => {
  // Ações do comando
});
```

Quando o comando precisa receber informações, utilizamos parâmetros:

```
Cypress.Commands.add('nomeDoComando', (parametro) => {
  // Utiliza o parâmetro recebido
});
```

4. Comandos de login

Login com credenciais válidas
No arquivo login.js:

```
Cypress.Commands.add('fazerLoginComCredenciaisValidas', () => {
  cy.get('#username').type('usuario');
  cy.get('#password').type('senha');
  cy.contains('button', 'Entrar').click();
});
```

Os seletores, as credenciais e o texto do botão são exemplos e devem corresponder à aplicação real.
Para utilizar o comando no teste:
cy.fazerLoginComCredenciaisValidas();

Assim, as etapas do login ficam centralizadas em um único lugar. Se o procedimento mudar, podemos atualizá-lo no comando personalizado.

Login com credenciais inválidas
Podemos criar outro comando no arquivo login.js:

```
Cypress.Commands.add('fazerLoginComCredenciaisInvalidas', () => {
  // Preencher credenciais inválidas
  // Clicar no botão Entrar
});
```

Uso no teste:
cy.fazerLoginComCredenciaisInvalidas();
O conteúdo interno deve reproduzir as etapas necessárias para testar o comportamento de login inválido.

5. Comando comum para verificar mensagens de Toast
Uma mensagem de Toast pode aparecer em diferentes funcionalidades, como login e transferência. Por isso, a validação pode ficar em common.js.

```
Cypress.Commands.add('verificarMensagemToast', mensagem => {
  cy.get('.toast').should('have.text', mensagem);
});
```

O parâmetro mensagem permite reutilizar o mesmo comando com textos diferentes.

Exemplo de uso:
cy.verificarMensagemToast('Autenticação necessária para transferências acima de R$5.000,00.');

A comparação com have.text é exata. Pontuação, espaços e formatação precisam corresponder ao texto exibido pela aplicação.

6. Comando para selecionar uma opção em uma combo box
Quando o procedimento de seleção se repete, podemos criar um comando que receba o nome do campo e a opção desejada.

No arquivo common.js:

```
Cypress.Commands.add(
  'selecionarOpcaoNaCombobox',
  (labelDoCampo, opcao) => {
    cy.get(`label[for="${labelDoCampo}"]`).parent().click();
    cy.contains(opcao).click();
  });
```

Exemplo de uso:
cy.selecionarOpcaoNaCombobox('conta-origem', 'Priscila');
cy.selecionarOpcaoNaCombobox('conta-destino', 'Julio');

Nesse exemplo:

- labelDoCampo: identifica o campo que será aberto.
- opcao: informa a opção que deve ser selecionada.
- A template string, delimitada por crases, permite inserir o valor de uma variável usando ${}.

Atenção: o exemplo pressupõe que o label esteja associado ao campo e que seu elemento pai seja clicável. A estrutura deve ser confirmada no HTML da aplicação. Se o componente for um <select> nativo, pode ser mais adequado utilizar o comando .select() do Cypress.

7. Comando específico para realizar uma transferência
Se os testes de transferência repetem as mesmas etapas e só mudam a conta de origem, a conta de destino e o valor, podemos centralizar esse fluxo em transferencias.js.

```
Cypress.Commands.add('realizarTransferencia',(contaOrigem, contaDestino, valor) => {
      cy.selecionarOpcaoNaCombobox('conta-origem',contaOrigem);
      cy.selecionarOpcaoNaCombobox('conta-destino',contaDestino);
      cy.get('#valor').clear().type(String(valor));
      cy.contains('button', 'Transferir').click();
   }
);
```

Exemplo de uso:
cy.realizarTransferencia('Priscila', 'Julio', 11);

Para testar uma transferência acima de R$ 5.000:
cy.realizarTransferencia('Priscila', 'Julio', 6000);

Os seletores e os textos devem ser adaptados à aplicação. A validação do resultado deve continuar no teste, usando a mensagem ou outra condição esperada.

8. Testando a regra de autenticação por valor
Podemos verificar se a aplicação exige autenticação para transferências acima de R$ 5.000.

Exemplo:

```
it('deve exigir autenticação para valores acima de 5000', () => {
   cy.fazerLoginComCredenciaisValidas();
   cy.realizarTransferencia('Priscila', 'Julio', 6000);
   cy.verificarMensagemToast('Autenticação necessária para transferências acima de R$5.000,00.');
});
```

Esse teste realiza as ações e verifica se a mensagem esperada aparece. Ele pressupõe que o login e os dados utilizados sejam válidos no ambiente de teste.

9. Como decidir onde colocar cada comando?

Arquivo	Responsabilidade
common.js	Comandos genéricos, como validar Toast e selecionar opções
login.js	Comandos exclusivos da funcionalidade de login
transferencias.js	Comandos exclusivos da funcionalidade de transferência

A decisão depende da finalidade do comando, não apenas do local em que ele foi criado. Um comando que pode ser útil em várias funcionalidades deve ficar em common.js; um fluxo específico deve ficar no arquivo da funcionalidade correspondente.

10. Benefícios dos Custom Commands

- Reduzem a repetição de código.
- Facilitam a manutenção dos testes.
- Tornam os testes mais curtos e legíveis.
- Permitem reutilizar ações em diferentes cenários.
- Centralizam procedimentos comuns.

Atenção: nem todo trecho repetido precisa virar um Custom Command. A abstração deve facilitar a leitura e a manutenção, e não esconder etapas simples sem necessidade.

11. Referências oficiais

- Cypress — Documentação: https://docs.cypress.io/
- Cypress — Custom Commands: https://docs.cypress.io/api/cypress-api/custom-commands
- Cypress — Organização dos testes: https://docs.cypress.io/app/core-concepts/writing-and-organizing-tests
- W3Schools — JavaScript Functions: https://www.w3schools.com/js/js_functions.asp
- W3Schools — JavaScript Template Strings: https://www.w3schools.com/js/js_string_templates.asp

12. O que preciso lembrar?

1. Cypress.Commands.add() cria comandos personalizados.
2. Parâmetros permitem reutilizar o comando com dados diferentes.
3. common.js concentra comandos genéricos.
4. Os arquivos de funcionalidades concentram comandos específicos.
5. O arquivo commands.js importa os módulos personalizados.
6. Custom Commands reduzem duplicação, mas as asserções continuam necessárias para verificar os resultados.

Conclusão: Custom Commands ajudam a organizar e reutilizar o código de automação, deixando os testes mais simples de ler e mais fáceis de manter.

---

22. CONFIGURANDO A GERAÇÃO DE RELATÓRIOS EM HTML

Relatórios:

- **Mochawesome**:
  - **Site**
  - **Instalação**: npm install --save-dev cypress-mochawesome-reporter

1. Por que utilizar um relatório HTML?
Ao executar os testes com npm test, o Cypress apresenta os resultados no terminal. Também é possível visualizar os testes na interface do próprio Cypress.

Entretanto, esses formatos podem ser técnicos para pessoas que não trabalham diretamente com automação de testes.

O Mochawesome gera um relatório HTML mais visual, que permite consultar:

- Quantidade de testes aprovados e reprovados.
- Tempo de execução dos testes.
- Detalhes dos erros.
- Capturas de tela (screenshots) dos testes que falharam.
- Capturas de tela adicionadas durante os testes.

2. Instalar o Mochawesome
No terminal, dentro da pasta do projeto, execute:
npm install --save-dev cypress-mochawesome-reporter

3. Configurar o cypress.config.js
Adicione a configuração do reporter:
reporter: "cypress-mochawesome-reporter",

```
reporterOptions: {
  reportDir: "cypress/reports/mochawesome",
  reportFilename: "mochawesome",
  html: true,
  json: true,
  overwrite: true,
  charts: true,
  embeddedScreenshots: true,
  saveAllAttempts: false,
  reportPageTitle: "Relatório de Testes Cypress",
},
```

Dentro de setupNodeEvents, registre o plugin:
require("cypress-mochawesome-reporter/plugin")(on);

Essa configuração permite gerar o relatório HTML e incorporar capturas de tela ao documento.

4. Configurar o arquivo cypress/support/e2e.js
Adicione a importação:
import 'cypress-mochawesome-reporter/register';
Essa linha registra o suporte do Mochawesome para a execução dos testes.

5. Executar os testes
Execute:
npm test
O Cypress executará os testes e o Mochawesome gerará o relatório HTML.

6. Visualizar o relatório

Com a configuração utilizada neste projeto, o arquivo gerado é:
cypress/reports/mochawesome/mochawesome.html

Abra esse arquivo no navegador para consultar os resultados.

7. Simular uma falha para entender o relatório

Na aula, o instrutor altera propositalmente um valor esperado em um teste de transferência para provocar uma falha.

O relatório passa a mostrar o teste aprovado e o teste reprovado, além dos detalhes do erro e da captura de tela correspondente.

Depois, é possível corrigir o valor e executar novamente os testes.

Resumo para revisão

- Cypress: executa os testes automatizados.
- Mochawesome: gera um relatório HTML visual.
- cypress.config.js: configura o reporter e registra o plugin.
- cypress/support/e2e.js: importa o suporte do Mochawesome.
- npm test: executa os testes e aciona a geração do relatório.

O objetivo da aula é transformar os resultados técnicos da execução em um relatório visual, mais fácil de consultar por desenvolvedores, profissionais de QA e pessoas de negócio.

---

23. CRIANDO O ARQUIVO README COM IA GENERATIVA

Etapas da atividade

1. Abra o VS Code e acesse **Extensões** (Ctrl + Shift + X).
2. Pesquise por **GitHub Copilot**, instale a extensão oficial e faça login na sua conta GitHub, caso seja solicitado.
3. Abra o Chat do Copilot pelo ícone na interface ou pelo menu **Exibir (View) → Chat**.

4. Abra a pasta do projeto 5-teste-web-cypress no VS Code para fornecer o contexto correto à IA.
5. Copie o prompt abaixo, cole no chat do Copilot e envie a solicitação.
6. Revise o README gerado, verificando se as informações correspondem aos arquivos reais do projeto.

Prompt para a IA

Crie um arquivo README para esse projeto de testes automatizados em Cypress e JavaScript.

Contexto:

- Todas as dependências do projeto estão no package.json.
- Usei automação com Cypress, organizando o código com Custom Commands.
- Os relatórios são gerados com cypress-mochawesome-reporter.
- Para que os testes sejam executados, a API e a aplicação web devem estar em execução:
    - API: https://github.com/MarianaCaetanoCosta/Mentoria-2.0-JulioDeLima/tree/main/Projetos/banco-api
    - Aplicação web: https://github.com/MarianaCaetanoCosta/Mentoria-2.0-JulioDeLima/tree/main/Projetos/banco-web

Regras:

- Descreva o objetivo do projeto, que é mostrar aos alunos da Mentoria 2.0 como automatizar testes com Cypress.
- Descreva os componentes que compõem o projeto.
- Forneça um guia de instalação.
- Documente os testes e os Custom Commands.
- Utilize os arquivos existentes como referência e não invente funcionalidades, testes ou comandos.

---

24. Referências oficiais

- Cypress: https://docs.cypress.io/
- Organização dos testes: https://docs.cypress.io/app/core-concepts/writing-and-organizing-tests
- Fixtures: https://docs.cypress.io/api/commands/fixture
- Custom Commands: https://docs.cypress.io/api/cypress-api/custom-commands
- Comando parent(): https://docs.cypress.io/api/commands/parent
- Comando contains(): https://docs.cypress.io/api/commands/contains
- Comando as(): https://docs.cypress.io/api/commands/as
- Boas práticas: https://docs.cypress.io/app/core-concepts/best-practices
- W3Schools — <select>: https://www.w3schools.com/tags/tag_select.asp
- W3Schools — Template Strings: https://www.w3schools.com/js/js_string_templates.asp

---

Conclusão do módulo

O objetivo é aprender a planejar testes, identificar problemas em aplicações web e automatizar cenários com Cypress. A evolução passa por escrever testes claros, verificar resultados com asserções, reduzir duplicação, organizar os dados, investigar falhas e documentar as execuções.

Para trabalhar com QA, o mais importante não é apenas saber executar comandos: é entender o comportamento esperado, escolher cenários relevantes, detectar defeitos e apresentar evidências confiáveis.
